

net-07 - Introdução ao Openstack Neutron

Contexto

Neste treinamento iremos abordar:

- Uma breve introdução ao Openstack Neutron
- Arquitetura e integração com o OVN
- Hands on - Instalação do Openstack via Devstack e integração com o OVN
- Hands on - Verificando os Security Groups no OVN
- Hands on - Analisando os arquivos de configuração do Neutron

Introdução ao Openstack Neutron

O OpenStack possui uma solução para gerenciar redes que é escalável, API-driven para a cloud e conforme outros componentes core do Openstack, a solução de rede pode ser usada por administradores e usuários.

O Neutron, é o nome do projeto do Openstack que gerencia os serviços de rede e complementando os outros serviços core do Openstack como Compute (Nova), Image (Glance), Identity (Keystone), Block (Cinder), Object (Swift) e Dashboard (Horizon) para prover uma solução completa de cloud.

O Neutron é exposto como uma application programmable interface (API) para os usuários e serviços internos da cloud e desta forma os usuários podem definir a conectividade de rede na cloud para seus workloads.

Quais as features são suportadas pelo Neutron?

As principais features suportadas pelo Neutron para que os usuários definam a conectividade de seus workloads são:

- **Switching** - Usando como backend uma solução de SDN (Software-Defined-Networking), é possível conectar os workloads em um mesmo segmento L2 (subnet)
- **Routing** - Para prover a conectividade dos workloads de diferentes subnets e com ambientes externos, o routing(rooteamento) é utilizado
- **ACL** - É possível criar ACLs (access control list) para permitir ou negar os acessos entrantes e saíntes para os workloads
- **Load Balancer** - Permite realizar o balanceamento de carga de serviços como HTTPS entre workloads internos da Cloud
- **Conectividade externa** - O acesso para a Internet é uma das features mais utilizadas, os workloads podem expor um serviço para a Internet ou mesmo acessar a Internet simplesmente para instalar um pacote ou clonar um repositório de git. Também é possível criar conectividade externa de forma privada com ambientes de Colocation do Datacenter ou mesmo usar uma solução de Interconnect para estabelecer conectividade privada com ambientes em cloud públicas.

Principais nomenclaturas dos recursos de rede do Neutron

O Neutron permite que os usuários tenham a habilidade de criar e configurar suas próprias subnets e conectar workloads. Os projetos do Openstack podem definir o uso de seu próprio endereçamento IP interno (RFC 1918) porém como boa prática sempre é necessário avaliar se futuramente irá existir conectividade privada com um ambiente externo e definir um padrão.

Existem dois tipos de redes que podem ser definidas no Openstack pelo Neutron:

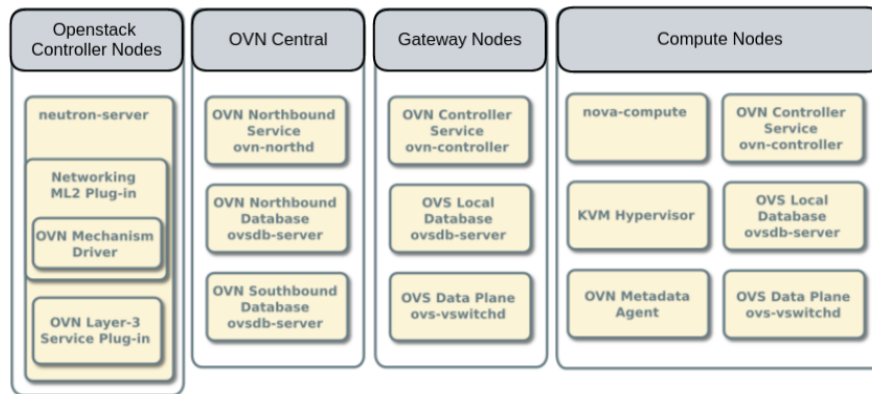
- **Project network** - Uma subnet criada por usuários ou administradores da cloud que irá ser utilizada para conectar os workloads e que em sua maioria é privada
- **Provider network** - Uma subnet criada somente por administradores da cloud que é utilizada para criar conectividade externa

Abaixo estão as principais nomenclaturas utilizadas:

- **Subnet** - Um bloco de endereços IP usado para conectar workloads e criar conectividade local e remota
- **Port** - Ponto de conexão de um workload em uma subnet. Alguns atributos que estão disponíveis são o MAC address e o endereço IP utilizado na subnet
- **Router** - É um objeto na cloud responsável em criar conectividade entre diferentes subnets de projeto e com a provider network
- **Security group** - Controla o tráfego entrante e saínte de uma porta através de ACLs
- **Ingress** - Tráfego de rede entrando no workload
- **Egress** - Tráfego de rede saindo do workload
- **DHCP** - Serviço de rede que gerencia o provisionamento dinâmico de IPs para os workloads
- **Metadata** - Serviço que é utilizado pelo cloud-init
- **Floating IP(FIP)** - É um endereço de IP público em sua maioria, que pode ser associado em um workload diretamente para que seja possível expor um serviço externamente. O endereço de Floating IP (FIP) não é visível pelo sistema operacional do workload
- **SNAT** - Para um router que esteja conectado em uma Provider network, por padrão será executado o SNAT (source network address translation) que visa criar conectividade dos workloads conectados em subnets privadas (RFC 1918) com a Internet na direção workload -> Internet.
- **Subnet Pool** - É uma maneira do Neutron escolher qual endereço de subnet disponível pode ser utilizada a partir de um Pool. O Pool nesse caso é uma subnet maior, por exemplo /16 e através do subnet pool, o usuário pode escolher subnets menores como /24. O subnet pool geralmente não é visível por todos os projetos do Openstack e é configurado através de uma regra de RBAC para dar essa visibilidade para um determinado projeto.

Arquitetura e integração com o OVN

O layout com todos os componentes da arquitetura se resume ao diagrama abaixo:



O Neutron que faz o papel de CMS, executa na camada de Controller do Openstack e possui vários componentes de serviço. O neutron-server por exemplo é responsável em interfacear todas as chamadas de API com os outros módulos da arquitetura do Openstack que solicitam algo de rede.

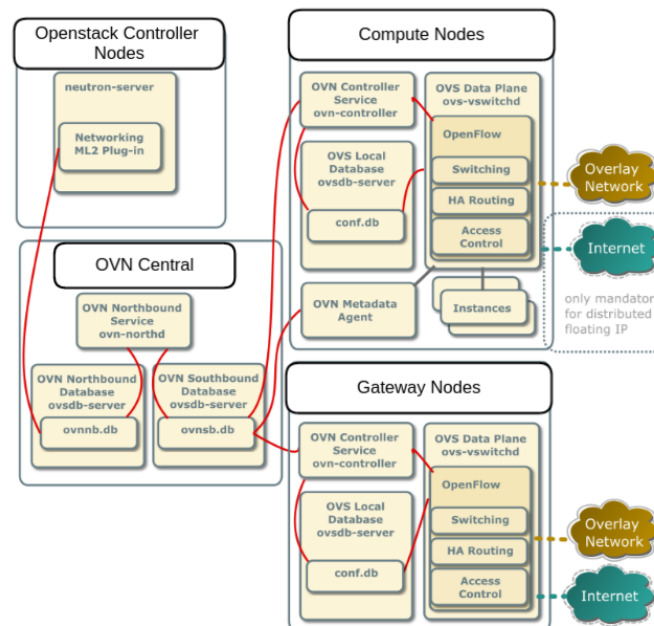
O Neutron se integra com N soluções de rede através do ML2, que é um framework que permite o Neutron ser híbrido. No caso do OVN, o Neutron possui o plugin ML2/OVN de integração e o principal propósito do plugin é traduzir toda a configuração lógica de rede armazenada no banco de dados do Neutron em um formato intermediário entendido pelo OVN que será o Logical Flow.

Existem algumas diferenças entre a integração tradicional do ML2/OVS com a ML2/OVN, abaixo detalhamos as principais:

	ML2/OVS	ML2/OVN
Communication	RabbitMQ	OVSDB
Metadata	o agente executa em conjunto com os agentes de DHCP e L3	o agente executa no Compute Node
L3	o agente executa nos Network Nodes e Compute Nodes(DVR)	Não existe agentes, tudo é manipulado pelo OVN e programado via Openflow flows nos Compute Nodes
DHCP	o agente executa nos Network Nodes	Não existe agente, o serviço é provido via Openflow flows nos Compute Nodes

Além de componentes nos Controller do Openstack, o Neutron possui o **metadata-agent** que está presente nos Chassis (Compute Nodes). Via serviço de Metadata o usuário final da Cloud pode configurar vários aspectos nas VMs: hostname, injetar chave SSH e etc. O metadata-agent é integrado com o OVN Southbound database para reportar alguns status mas também interage com o serviço do Nova do Openstack.

No diagrama abaixo temos uma visão macro dos componentes do OVN e sua relação com o Neutron:

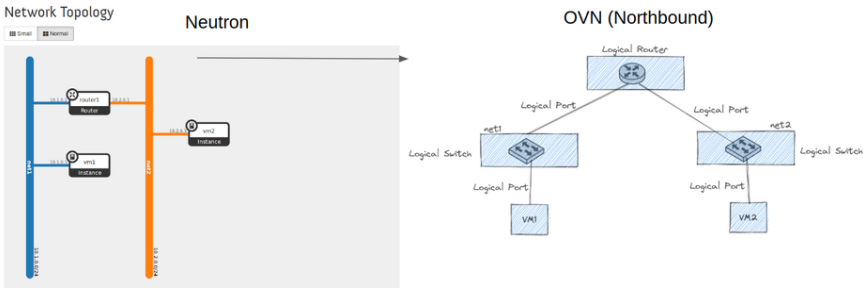


Como podem observar o metadata-agent está integrado com o OVS, que irá instanciar por subnet (Logical Switch) um namespace com o **haproxy** para que as instâncias consigam atingir o endereço 169.254.169.254 e fazer o download do **cloud-init** que terá as configurações que o usuário final realizou.

Podemos fazer um de-para entre a nomenclatura de rede utilizada pelo Neutron com os Logical Flows do OVN:

Neutron	OVN - Logical Flow
Network	Logical Switch
Subnet	
Router	Logical Router
Port	Logical Switch Port ou Logical Router Port
Security Group/Rule	Port Group/ACL
Floating IP	NAT (dnat_and_snat)

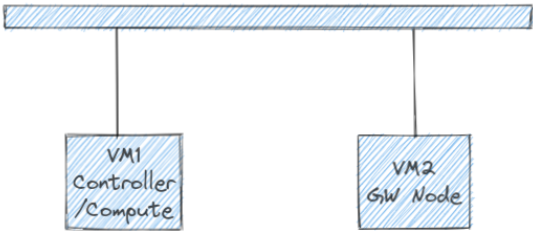
Abaixo temos um de-para visual de uma típica topologia de rede:



Instalação do Openstack via Devstack e integração com o OVN

- Para fins de teste **apenas**, se recomenda ter o seguinte setup:
- Controller/ComputeNode - uma VM executando o Ubuntu 20.04 com 4 vCPUs/8GB de ram/40G de disco
 - Gateway Node - uma VM executando o Ubuntu 20.04 com 1 vCPU/2GB de ram/20G de disco
 - ambas presentes na mesma subnet
 - com os pacotes latest instalados para seguir os exercícios.

A topologia de rede desse exercício deve ser parecida com a abaixo:



Crie um plano de endereçamento conforme seu setup se comparando com o abaixo:

VM	nic0
controller/compute	10.0.10.100/24
gwnode1	10.0.10.101/24

1 - Instalando o Openstack via Devstack no modo All-in-One e integrando com o OVN

Siga os passos abaixo para executar o setup All-in-one:

```
1 # add-apt-repository -y cloud-archive:yoga
2 # useradd -s /bin/bash -d /opt/stack -m stack
3 # echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
4 # su - stack
5 $ git clone https://opendev.org/openstack/devstack
6 $ cd devstack
```

```

7 $ git branch -a
8 $ git checkout stable/yoga
9
10 ###Crie o arquivo local.conf###
11 $ vi local.conf
12 [[local|localrc]]
13 DATABASE_PASSWORD=password
14 RABBIT_PASSWORD=password
15 SERVICE_PASSWORD=password
16 SERVICE_TOKEN=password
17 ADMIN_PASSWORD=password
18 Q_AGENT=ovn
19 Q_ML2_PLUGIN_MECHANISM_DRIVERS=ovn,logger
20 Q_ML2_PLUGIN_TYPE_DRIVERS=local,flat,vlan,geneve
21 Q_ML2_TENANT_NETWORK_TYPE="geneve"
22 LOGFILE=$DEST/logs/stack.sh.log
23 enable_service ovn-northd
24 enable_service ovn-controller
25 enable_service q-ovn-metadata-agent
26 enable_service q-svc
27 disable_service q-agt
28 disable_service q-l3
29 disable_service q-dhcp
30 disable_service q-meta
31 enable_plugin neutron https://opendev.org/openstack/neutron stable/yoga
32 enable_service q-trunk
33 enable_service q-dns
34 enable_service q-port-forwarding
35 enable_service q-qos
36 enable_service neutron-segments
37 enable_service q-log
38 enable_service horizon
39 enable_service cinder c-sch c-api c-vol
40 OVN_BUILD_MODULES=True
41 PROVIDER_NETWORK_TYPE=flat
42 ENABLE_CHASSIS_AS_GW=True
43 [[post-config|$NOVA_CONF]]
44 [scheduler]
45 discover_hosts_in_cells_interval = 2
46
47 ###Inicie a instalação####
48 $ ./stack.sh

```

Após a instalação ser finalizada, você receberá as informações de acesso ao Horizon:

```

1 This is your host IP address: 10.0.10.100
2 This is your host IPv6 address: ::1
3 Horizon is now available at http://10.0.10.100/dashboard
4 Keystone is serving at http://10.0.10.100/identity/
5 The default users are: admin and demo
6 The password: XXXXXXXXXX

```

Caso ocorra algum problema durante a instalação do Devstack, por exemplo:

```

1 ERROR: AppArmor parser error for /etc/apparmor.d/usr.sbin.libvirtd in /etc/apparmor.d/usr.sbin.libvirtd at line 29: Invalid capability bpf.

```

Execute a rotina abaixo para removê-lo:

```

1 $ ./unstack.sh

```

Altera o arquivo abaixo comentando os parâmetros indicados:

```

1 $ sudo vi /etc/apparmor.d/usr.sbin.libvirtd
2 #####Comente
3 #capability bpf,
4 #capability perfmon,

```

Execute novamente o processo de instalação:

```

1 $ ./stack.sh

```

2 - Ativando o Gateway Node

Durante a instalação do Openstack via **Devstack**, os serviços do OVN Central foram instalados e configurados porém o setup considerou o compute node como Gateway Node(Chassi Gateway).

Vamos ajustar o setup e incluir a segunda VM com o papel de Gateway Node e remover esse papel do compute node.

Na VM de Gateway Node, faça a instalação dos pacotes necessários conforme abaixo:

```
1 # add-apt-repository -y cloud-archive:yoga
2 # apt install -y ovn-host openvswitch-switch
```

Configure os parâmetros abaixo do Gateway Node e aponte o IP da Controller do OVN para o IP conforme seu setup:

```
1 # ovs-vsctl --no-wait set open_vswitch . external-ids:hostname=ovn-sandbox2
2 # ovs-vsctl --no-wait set open_vswitch . external-ids:ovn-remote=tcp:10.0.10.100:6642 #Altere pelo IP da Controller/Compute
3 # ovs-vsctl --no-wait set-manager ptcp:6640:127.0.0.1
4 # ovs-vsctl --no-wait set open_vswitch . external-ids:ovn-bridge=br-int
5 # ovs-vsctl --no-wait set open_vswitch . external-ids:ovn-encap-type=geneve
6 # ovs-vsctl --no-wait set open_vswitch . external-ids:ovn-encap-ip=10.0.10.200 #Altere pelo IP local do Gateway Node
7 # ovs-vsctl --no-wait set open . external-ids:ovn-cms-options="enable-chassis-as-gw"
```

Mais detalhes dos parâmetros configurados acima:

- **hostname** - nome amigável para identificar o Chassi
- **ovn-remote** - endereço IP do OVN Central (no modo cluster, existirão 3 entradas)
- **ovn-bridge** - bridge de integração gerenciada pelo OVS
- **ovn-encap-type** - protocolo de encapsulamento utilizado
- **ovn-encap-ip** - endereço IP de origem para os túneis
- **ovn-cms-options** - configura esse Chassi como Gateway Node

Crie a bridge **br-ex** que irá prover a conectividade externa:

```
1 # ovs-vsctl --no-wait -- --may-exist add-br br-ex -- set bridge br-ex protocols=OpenFlow13,OpenFlow15
2 # ovs-vsctl --no-wait br-set-external-id br-ex bridge-id br-ex
3 # ovs-vsctl set open . external-ids:ovn-bridge-mappings=public:br-ex
4 # ip addr add 172.24.4.1/24 dev br-ex
5 # ip link set br-ex up
```

Valide as configurações:

```
1 # ovs-vsctl list open_vswitch
2 _uuid          : 4f81f863-0e84-4ae6-ad08-010af9855e90
3 bridges        : [6acd50c-a885-43ae-895f-1c42307960db, 93531392-06ef-4a45-a6cf-530f4e47836c]
4 cur_cfg        : 1
5 datapath_types : [netdev, system]
6 datapaths      : {system=9ef114be-6af3-4bbc-ab35-7c3606d46679}
7 db_version     : "8.3.0"
8 dpdk_initialized : false
9 dpdk_version   : none
10 external_ids   : {hostname=ovn-sandbox2, ovn-bridge=br-int, ovn-bridge-mappings="public:br-ex", ovn-cms-options=enable-chassis-as-gw, ovn-encap-
11 iface_types    : [bareudp, erspan, geneve, gre, gtpu, internal, ip6erspan, ip6gre, lisp, patch, stt, system, tap, vxlan]
12 manager_options : [32d8d146-0994-4938-8e84-0f9179a200b7]
13 next_cfg       : 1
14 other_config    : {vlan-limit="0"}
15 ovs_version     : "2.17.0"
16 ssl            : []
17 statistics     : {}
18 system_type     : ubuntu
19 system_version  : "20.04"
```

No Controller/Compute, teremos que remover a configuração de Gateway Node que foi configurada durante o setup via Devstack:

```
1 $ sudo ovs-vsctl remove open . external-ids ovn-cms-options="enable-chassis-as-gw"
2 $ sudo ovs-vsctl remove open . external-ids ovn-bridge-mappings
3 $ sudo ip addr del 172.24.4.1/24 dev br-ex
4 $ sudo ip link set br-ex down
5 $ sudo ovs-vsctl del-br br-ex
```

3 - Realizando uma validação no setup

Primeiramente, a partir do Controller/Compute onde temos os serviços do Openstack e do OVN Central instalados siga os seguintes passos:

Checar se temos 2 Chassis no OVN Southbound:

```
1 # ovn-sbctl list chassis
2 _uuid          : 5ffbc392-87b9-4e97-b6fc-1f9d8f71e5e6
3 encaps         : [69ba3a62-79dd-4304-9980-06efe22b1ae1]
4 external_ids   : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", i
5 hostname       : ovn-sandbox2
6 name           : "0727afaf-5048-4624-a898-1181367f0233"
7 nb_cfg         : 0
8 other_config    : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", i
9 transport_zones : []
```

```

10 vtep_logical_switches: []
11
12 _uuid          : b283613e-8805-4bc0-a757-3c4897f9349f
13 encaps         : [3c5de98a-4e23-4eb8-b8b1-95b65c980692]
14 external_ids   : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", i
15 hostname       : ovn-sandbox1
16 name           : "cab023cb-ba95-4ec3-aa9b-61c2267833cd"
17 nb_cfg         : {}
18 other_config   : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", i
19 transport_zones : []
20 vtep_logical_switches: []

```

O output acima nos informa que temos 2 Chassis, um deles é o Gateway Node que tem a opção `ovn-cms-options=enable-chassis-as-gw` configurada.

Usando as credenciais de admin do Openstack, verifique os agents de rede habilitados:

```

1 $ source openrc admin
2 $ openstack network agent list
3
4 +-----+-----+-----+-----+-----+-----+-----+
5 | ID | Agent Type | Host | Availability Zone | Alive | State | Binary |
6 +-----+-----+-----+-----+-----+-----+-----+
7 | fabc045a-d8e4-565b-a958-55eba692c152 | OVN Metadata agent | ovn-sandbox1 | | :- ) | UP | neutron-ovn-metadata-agent |
8 | cab023cb-ba95-4ec3-aa9b-61c2267833cd | OVN Controller agent | ovn-sandbox1 | | :- ) | UP | ovn-controller |
9 | 0727afaf-5048-4624-a898-1181367f0233 | OVN Controller Gateway agent | ovn-sandbox2 | | :- ) | UP | ovn-controller |
10 +-----+-----+-----+-----+-----+-----+-----+

```

No output acima, podemos verificar que os metadata agents e ovn-controllers estão listados. No caso do Gateway Node, o mesmo não possui um agent de metadata.

Durante o setup via Devstack, foram criadas algumas networks no Neutron automaticamente:

```

1 $ openstack network list
2
3 +-----+-----+-----+-----+-----+
4 | ID | Name | Subnets |
5 +-----+-----+-----+-----+-----+
6 | 4bc227b3-aad7-4350-bf7b-af7838910261 | public | 54b59f46-8093-4a3b-b66e-2c5e3d305193, b6659be8-4e6d-4979-a84f-e3960f2d79da |
7 | 5bb5b9ad-7348-474f-9e14-66f34278e373 | private | 229329b1-1627-49e2-bfb3-deacba7155f9, f3a990c6-e32b-4746-9983-fb5bf42020c4 |
8 | dded64ce-9130-4f95-a043-81b00421f9d5 | shared | 72eb895f-38da-47d9-b908-afb21ff9c811 |
9 +-----+-----+-----+-----+-----+

```

Uma das networks é uma Network Provider que irá permitir atribuir Floating IPs para os routers e VMs.

Também foi criado um router:

```

1 $ openstack router list
2
3 +-----+-----+-----+-----+-----+
4 | ID | Name | Status | State | Project |
5 +-----+-----+-----+-----+-----+
6 | d18829fb-2cf8-47b7-85d6-0e96caf2e90f | router1 | ACTIVE | UP | 6a775cfe510a4b8996fdd67d0343b4f0 |
7 +-----+-----+-----+-----+-----+

```

Do lado do OVN Northbound, poderemos verificar que existem Logical Flows correspondentes para o que existe no Neutron:

```

1 # ovn-nbctl show
2 switch a40f61bc-6e32-48f9-919c-1d6a66bce95c (neutron-dded64ce-9130-4f95-a043-81b00421f9d5) (aka shared)
3   port 4b182d83-8912-46f5-84b7-d9827c2d261d
4     type: localport
5     addresses: ["fa:16:3e:c7:ad:bb 192.168.233.2"]
6 switch eff9b504-4a40-4ad8-965d-f33cc52a52c8 (neutron-4bc227b3-aad7-4350-bf7b-af7838910261) (aka public)
7   port cf47f4af-5cfd-4293-8254-1573856157c1
8     type: router
9     router-port: lrp-cf47f4af-5cfd-4293-8254-1573856157c1
10    port 22571ee6-3420-4d09-8feb-69f9e8fdf22e
11      type: localport
12      addresses: ["fa:16:3e:5b:4e:84"]
13    port provnet-f62f29e7-afc6-467a-97b0-1d3988477b46
14      type: localnet
15      addresses: ["unknown"]
16 switch 4830c62f-7de6-4c6d-91aa-6eade80c7291 (neutron-5bb5b9ad-7348-474f-9e14-66f34278e373) (aka private)
17   port 286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
18     type: router
19     router-port: lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
20    port 7b399866-1eb0-4654-9f7b-04240ec12e2a
21      type: localport
22      addresses: ["fa:16:3e:3d:c6:9e 10.0.0.2 fd42:4193:f777:0:f816:3eff:fe3d:c69e"]
23    port 7bbb8967-b1dd-475e-b9ec-e6aba3931183
24      type: router
25      router-port: lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
26 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)

```

```

27     port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
28         mac: "fa:16:3e:71:b4:1f"
29         networks: ["172.24.4.147/24", "2001:db8::1/64"]
30         gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
31     port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
32         mac: "fa:16:3e:03:e6:64"
33         networks: ["fd42:4193:f777::1/64"]
34     port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
35         mac: "fa:16:3e:8f:c6:8b"
36         networks: ["10.0.0.1/26"]
37     nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
38         external ip: "172.24.4.147"
39         logical ip: "10.0.0.0/26"
40         type: "snat"

```

Com o ID do router do Neutron, podemos avaliar o ID correspondente no OVN Northbound:

```

1 # ovn-nbctl show | grep d18829fb-2cf8-47b7-85d6-0e96caf2e90f
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)

```

Verificando o output acima, sabemos que o ID do router "**router1**" é o `f6b46d50-fbdc-495f-a195-0bf4a92be2be`. Desse modo, podemos checar mais detalhes da topologia de rede desse router:

```

1 # ovn-nbctl show f6b46d50-fbdc-495f-a195-0bf4a92be2be
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)
3     port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
4         mac: "fa:16:3e:71:b4:1f"
5         networks: ["172.24.4.147/24", "2001:db8::1/64"]
6         gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
7     port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
8         mac: "fa:16:3e:03:e6:64"
9         networks: ["fd42:4193:f777::1/64"]
10    port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
11        mac: "fa:16:3e:8f:c6:8b"
12        networks: ["10.0.0.1/26"]
13    nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
14        external ip: "172.24.4.147"
15        logical ip: "10.0.0.0/26"
16        type: "snat"

```

Faça a criação de uma VM no projeto **demo**, com o nome de **vm1**, conectando na network **private**, atribua um Floating IP e volte a verificar o router:

```

1 # ovn-nbctl show f6b46d50-fbdc-495f-a195-0bf4a92be2be
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)
3     port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
4         mac: "fa:16:3e:71:b4:1f"
5         networks: ["172.24.4.147/24", "2001:db8::1/64"]
6         gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
7     port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
8         mac: "fa:16:3e:03:e6:64"
9         networks: ["fd42:4193:f777::1/64"]
10    port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
11        mac: "fa:16:3e:8f:c6:8b"
12        networks: ["10.0.0.1/26"]
13    nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
14        external ip: "172.24.4.147"
15        logical ip: "10.0.0.0/26"
16        type: "snat"
17    nat 8db7a3ed-2897-41ce-8fa6-3a795e540a3b
18        external ip: "172.24.4.96"
19        logical ip: "10.0.0.42"
20        type: "dnat_and_snat"

```

Repare que após atribuirmos um Floating IP para a nova VM, temos uma regra de NAT (dnat_and_snat) criada.

Acessando a console da VM, você irá perceber que o hostname definido durante a criação da VM foi automaticamente configurado. Isso devido ao serviço do metadata estar funcionando:

```

1 Connected to domain 'instance-00000002'
2 Escape character is ^] (Ctrl + ])
3
4 login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
5 vm1 login:

```

Anteriormente foi comentado que o serviço de metadata acaba criando um namespace para cada subnet criada:

```
1 $ sudo ip netns
2 ovnmeta-5bb5b9ad-7348-474f-9e14-66f34278e373 (id: 0)
```

O namespace acima está executando o seguinte processo do haproxy:

```
1 $ ps aux | grep ovn-metadata-proxy
2 stack      552140  0.0  0.0 236624 3920 ?        Ssl  13:36   0:03 haproxy -f /opt/stack/data/neutron/ovn-metadata-proxy/5bb5b9ad-7348-474f-9e14-66f34278e373
3 stack      570322  0.0  0.0   5196   660 pts/1    R+   18:05   0:00 grep --color=auto ovn-metadata-proxy
4
```

Verificando a configuração do haproxy:

```
1 $ cat /opt/stack/data/neutron/ovn-metadata-proxy/5bb5b9ad-7348-474f-9e14-66f34278e373.conf
2
3 global
4     log             /dev/log local0 debug
5     user            stack
6     group           stack
7     maxconn         1024
8     pidfile         /opt/stack/data/neutron/external/pids/5bb5b9ad-7348-474f-9e14-66f34278e373.pid.haproxy
9     daemon
10
11 defaults
12     log global
13     mode http
14     option httplog
15     option dontlognull
16     option http-server-close
17     option forwardfor
18     retries         3
19     timeout http-request 30s
20     timeout connect  30s
21     timeout client   32s
22     timeout server   32s
23     timeout http-keep-alive 30s
24
25 listen listener
26     bind 169.254.169.254:80
27     server metadata /opt/stack/data/neutron/metadata_proxy
28     http-request add-header X-OVN-Network-ID 5bb5b9ad-7348-474f-9e14-66f34278e373
```

Analise a configuração de rede do namespace:

```
1 $ sudo ip netns exec ovnmeta-5bb5b9ad-7348-474f-9e14-66f34278e373 ip a
2 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4     inet 127.0.0.1/8 scope host lo
5         valid_lft forever preferred_lft forever
6     inet6 ::1/128 scope host
7         valid_lft forever preferred_lft forever
8 2: tap5bb5b9ad-71@if19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
9     link/ether fa:16:3e:3d:c6:9e brd ff:ff:ff:ff:ff:ff link-netnsid 0
10    inet 169.254.169.254/32 brd 169.254.169.254 scope global tap5bb5b9ad-71
11        valid_lft forever preferred_lft forever
12    inet 10.0.0.2/26 brd 10.0.0.63 scope global tap5bb5b9ad-71
13        valid_lft forever preferred_lft forever
14
```

Conforme o output acima, é possível verificar que o namespace possui uma interface tap com o IP 10.0.0.2 (segundo IP da subnet 10.0.0.0/26) e um endereço adicional 169.254.169.254/32.

Voltando ao console da VM criada, verifique a tabela de roteamento:

```
1 $ ip route
2 default via 10.0.0.1 dev eth0
3 10.0.0.0/26 dev eth0 scope link src 10.0.0.42
4 169.254.169.254 via 10.0.0.2 dev eth0
```

Podemos determinar que existe uma rota para o IP 169.254.169.254 via o IP 10.0.0.2 (namespace do metadata). Obs: Essa VM recebeu sua configuração de rede via DHCP.

Verifique a conectividade com o gateway da rede Provider (172.24.4.1):

```
1 $ ping -c 3 172.24.4.1
2 PING 172.24.4.1 (172.24.4.1): 56 data bytes
3 64 bytes from 172.24.4.1: seq=0 ttl=63 time=1.408 ms
4 64 bytes from 172.24.4.1: seq=1 ttl=63 time=2.244 ms
5 64 bytes from 172.24.4.1: seq=2 ttl=63 time=1.051 ms
```

Verificando os Security Groups no OVN [↗](#)

Neste exercício, vamos analisar o **Security Group** que é responsável em permitir ou negar o tráfego de ingress ou egress para determinadas portas. Ele é composto por rules, que são responsável em manipular os pacotes baseado em critérios como:

- Direção dos pacotes (Ingress ou Egress)
- IP Origem/Destino
- Protocolo (Ex: ICMP/TCP/UDP)
- Porta de serviço (Ex: 22, 443)

Para verificarmos o Security Group que uma VM está usando, primeiramente verifique no projeto quais estão disponíveis:

```
1 $ openstack security group list --project demo
2 +-----+-----+-----+-----+-----+
3 | ID | Name | Description | Project | Tags |
4 +-----+-----+-----+-----+-----+
5 | fed05084-5a95-4054-87e3-794fca4f7075 | default | Default security group | 6a775cfe510a4b8996fdd67d0343b4f0 | [] |
6 +-----+-----+-----+-----+-----+
```

Verifique o ID da VM e encontre o Security Group associado:

```
1 $ openstack server list
2 +-----+-----+-----+-----+-----+
3 | ID | Name | Status | Networks | Image |
4 +-----+-----+-----+-----+-----+
5 | dac8d874-1aa5-494c-88a6-234fd1575201 | vm1 | ACTIVE | private=10.0.0.42, 172.24.4.96, fd42:4193:f777:0:f816:3eff:fe77:1296 | cirros-0.5.2-x86_64-d
6 +-----+-----+-----+-----+-----+
7 $ openstack server show dac8d874-1aa5-494c-88a6-234fd1575201 -f json -c security_groups
8 {
9   "security_groups": [
10    {
11     "name": "default"
12    }
13  ]
14 }
```

Liste as rules desse Security Group:

```
1 $ openstack security group rule list fed05084-5a95-4054-87e3-794fca4f7075
2 +-----+-----+-----+-----+-----+-----+-----+-----+
3 | ID | IP Protocol | Ethertype | IP Range | Port Range | Direction | Remote Security Group | Remote
4 +-----+-----+-----+-----+-----+-----+-----+-----+
5 | 2bb07f1b-488e-48f7-b8f8-f548522b9614 | None | IPv6 | ::/0 | | ingress | fed05084-5a95-4054-87e3-794fca4f7075 | None
6 | 36a1df35-7edd-4041-b02b-90b75f9839dc | None | IPv4 | 0.0.0.0/0 | | egress | None | None
7 | 6a4e7221-d8a0-40a0-a2dc-b42001312747 | None | IPv4 | 0.0.0.0/0 | | ingress | fed05084-5a95-4054-87e3-794fca4f7075 | None
8 | 81adc3c1-3181-4788-8d3f-1547649f4c7d | None | IPv6 | ::/0 | | egress | None | None
9 | deff4bfa-faa7-48eb-855b-69e0c703064b | tcp | IPv4 | 0.0.0.0/0 | 22:22 | ingress | None | None
10 +-----+-----+-----+-----+-----+-----+-----+-----+
```

Veja que além das rules que são criadas por padrão, foi adicionado uma rule de ingress para a porta 22.

Usando o ID do Security Group, é possível verificar sua correspondente Port Group no OVN:

```
1 # ovn-nbctl --bare --columns=name,ports list Port_Group
2 neutron_pg_drop
3 955fbfb8-bd0b-4ec8-b4aa-916b398120f1
4
5 pg_c22ba31d_a6fb_43b8_a3b3_f05d5196055d
6
7
8 pg_a89d2d52_23a4_4e4e_a67d_42665d6041b2
9
10
11 pg_fed05084_5a95_4054_87e3_794fca4f7075
12 955fbfb8-bd0b-4ec8-b4aa-916b398120f1
```

Verificando o output acima, o ID `pg_fed05084_5a95_4054_87e3_794fca4f7075` é o Security group `fed05084-5a95-4054-87e3-794fca4f7075` do Openstack.

Na terminologia do OVN, PG é Port Group que podemos comparar ao Security Group do Openstack e dentro do mesmo estão as ACLs criadas:

```
1 # ovn-nbctl acl-list pg_fed05084_5a95_4054_87e3_794fca4f7075
2 from-lport 1002 (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4) allow-related
3 from-lport 1002 (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6) allow-related
4 to-lport 1002 (outport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == $pg_fed05084_5a95_4054_87e3_794fca4f7075_ip4) allow-related
5 to-lport 1002 (outport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == 0.0.0.0/0 && tcp && tcp.dst == 22) allow-related
6 to-lport 1002 (outport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6 && ip6.src == $pg_fed05084_5a95_4054_87e3_794fca4f7075_ip6) allow-related
```

Analisando os arquivos de configuração do Neutron

Neste exercício, vamos verificar os arquivos relacionados com a integração entre o Neutron e OVN Central.

Em nosso setup do Openstack e no nível da Controller, iremos analisar primeiramente o arquivo **`/etc/neutron/neutron.conf`** :

```
1 $ cat /etc/neutron/neutron.conf | grep ovn
2 service_plugins = ovn-router, trunk, qos, log, segments, port_forwarding
```

O arquivo `neutron.conf` possui as configurações globais que o Neutron suportará. No parâmetro `service_plugins`, é possível determinar as features de rede que o Neutron irá habilitar.

 O setup utilizado All-in-One não prevê a configuração do OVN em cluster

Um outro arquivo muito importante é o de plugin para integração via ML2/OVN, **`/etc/neutron/plugins/ml2/ml2_conf.ini`** :

```
1 [ml2]
2 tenant_network_types = geneve
3 extension_drivers = port_security, qos, dns_domain_keywords
4 type_drivers = local, flat, vlan, geneve
5 mechanism_drivers = ovn, logger
6
7 [ml2_type_flat]
8 flat_networks = public,
9
10 [ml2_type_geneve]
11 max_header_size = 38
12 vni_ranges = 1:65536
13
14 [ml2_type_vlan]
15 network_vlan_ranges = public
16
17 [ml2_type_vxlan]
18 vni_ranges = 1:1000
19
20 [securitygroup]
21 enable_security_group = True
22
23 [ovn]
24 dns_servers = 8.8.8.8
25 ovn_metadata_enabled = True
26 ovn_l3_scheduler = leastloaded
27 neutron_sync_mode = log
28 ovn_sb_connection = tcp:10.0.10.100:6642
29 ovn_nb_connection = tcp:10.0.10.100:6641
```

Do lado do compute node, podemos verificar a configuração do serviço de metadata no arquivo **`/etc/neutron/neutron_ovn_metadata_agent.ini`** :

```
1 [ovn]
2 ovn_sb_connection = tcp:10.0.10.100:6642
```

Conclusão

Neste treinamento abordamos na prática a integração entre Neutron e OVN através de um deployment do Openstack via Devstack. Fizemos um paralelo entre a nomenclatura de rede utilizada no Neutron com a do OVN e entendemos vários aspectos técnicos entre Neutron e OVN.

 Para dar sequência no próximo treinamento, não remova as configurações desse setup.